Parallel Programming for Computer Graphics

Mid-term project: *2D Circle Rasterization*

Francesco Acciaioli
University of Florence
francesco.acciaioli@stud.unifi.it

Abstract

Rasterizing large numbers of alpha-blended, depth-ordered 2D primitives is a core building block of real-time and offline rendering, and it is easily parallel in principle but easy to implement inefficiently. This report presents a CPU-only 2D circle rasterizer, implemented in Python with Numba, that draws N alpha-blended circles onto an $H \times W$ image using the painter's algorithm. Parallelism comes from Python threads running on the free-threaded (GIL-free) build. Four kernels of increasing complexity are compared: a sequential baseline, a naive per-pixel parallel kernel, a tile-based parallel kernel with bounding-box culling, and a tile-based kernel additionally written for auto-vectorization (branchless SIMD inner loop). On a 4-core / 8-thread AMD Ryzen 5 2400G, the tiled, vectorized kernel reaches a maximum speedup of [$S_{\max}$] over the sequential baseline (on [p] threads), while a dedicated scalar-vs-SIMD comparison shows vectorization alone contributes roughly [X×] at a fixed thread count. A broader set of micro-benchmarks additionally isolates the effect of resolution, circle count, tile size, memory layout (SoA vs. AoS) and weak scaling, providing a systematic picture of where the performance comes from and where it saturates.

1. Introduction

This report presents the design, implementation and performance analysis of a sequential and parallelized 2D circle rasterizer. The project was developed to demonstrate how coarse-grained multithreading and fine-grained vectorization can be combined to accelerate a classic computer-graphics workload.

1.1. Problem Description and Motivation

Rasterization is the process of converting geometric elements into pixels. This project targets a deliberately simple but representative primitive set: N circles, each defined by a 2D center, a depth coordinate z, a radius, an RGB color and an alpha value (opacity). Circles are composited onto the image using the *painter's algorithm*: they are sorted by increasing depth and drawn back-to-front, blending each new circle onto the frame buffer with the standard "over" operator

$$\text{out} = \text{src}_{\text{rgb}} \cdot a + \text{dst}_{\text{rgb}} \cdot (1 - a).$$

The name comes from the way a painter works: the background is painted first and nearer objects are painted over it, so whatever is closer covers whatever lies behind, without a depth buffer. With transparency the order is essential, because the "over" operator is not commutative: two overlapping circles blended in opposite orders give different colors. This is the same compositing model used by real rendering pipelines (e.g. particle systems, 2D vector graphics, UI compositors), which makes the problem a realistic stand-in for a much larger class of rendering workloads.

1.2. Computational Challenges

A pixel (i, j) belongs to circle k if its center $(i+0.5,\ j+0.5)$ satisfies $dx^2 + dy^2 \leq r_k^2$. In the naive formulation every one of the $H \times W$ pixels must be tested against every one of the N circles, giving $O(H \cdot W \cdot N)$ work per frame. Even after restricting each circle to its axis-aligned bounding box, the workload remains large and highly *irregular*: circles vary enormously in radius (a few pixels up to $\sim 12\%$ of the image side), so the amount of work associated with different pixels is not uniform. This irregularity, rather than raw arithmetic volume, is the central challenge that the parallelization strategy has to address.

1.3. Objectives and Scope

The goal of the project is to explore, in a controlled way, which parallelization and vectorization techniques are effective for this class of workload, and why. The scope includes:

1. **Sequential implementation:** a straightforward, single-threaded kernel used as the correctness and performance baseline.
2. **Parallel optimization:** two coarse-grained multi-threaded strategies (per-pixel-row and per-tile), plus a tile-based kernel whose inner loop is written to be auto-vectorized by the compiler.
3. **Systematic micro-benchmarking:** isolating, one variable at a time, the effect of thread count, image resolution, circle count, tile size, SIMD, memory layout and weak scaling.

2. Algorithm Description

2.1. Data Structure

Circles are stored in a *Structure-of-Arrays* (SoA) container (`CircleSoA`): eight parallel, contiguous `float32` NumPy arrays $(x, y, z, \text{radius}, r, g, b, a)$, rather than a list of per-circle objects. This layout is cache-friendly and is the natural starting point for vectorization, since each attribute is unit-stride in memory [3]. Sorting by depth is done once, up front, with a *stable* mergesort, so that circles with equal z keep their original (insertion) order deterministically; the sort is never repeated inside a rendering kernel.

2.2. Sequential Algorithm

The baseline kernel loops over circles from far to near. For each circle it computes an integer bounding box clipped to the image, then scans only the pixels inside that box, testing the circle equation and applying the "over" blend in place:

```
for k in range(n): # far -> near
   x0,y0,x1,y1 = bbox(k) # clipped to image
   for j in range(y0, y1):
      for i in range(x0, x1):
         if inside_circle(i, j, k):
            blend_over(img[j, i], color(k))
```

This bounding-box restriction is the only optimization present in the sequential version; it is still $O(N)$ circles times, on average, $O(r_k^2)$ pixels, and it writes directly into a shared image buffer, which is safe here only because the loop is single-threaded.

2.3. Parallelization Strategy

Two independent opportunities for parallelism were identified:

Parallelizing across the image (multithreading). Because the final color of a pixel does not depend on the color of any other pixel (circles are looked up, not accumulated across pixels), the image can be partitioned into disjoint regions — rows or square tiles — and each region processed independently by a different thread, with no shared mutable state and therefore no locks.

The painter's algorithm also constrains how the work can be split: every pixel must receive its circles in depth order. Parallelizing over circles would let different threads blend onto the same pixel in an arbitrary order, requiring locks or a reordering step. Parallelizing over pixels keeps the order for free: each thread owns its pixels and walks the already sorted circle list, which is why sorting is done once, before the threads start.

Vectorizing the inner loop (SIMD). Inside a tile, testing a circle against many pixels performs the same arithmetic (subtraction, multiplication, comparison, blend) on independent data. If that inner loop is written without early-exit branches and with a small, contiguous, per-tile buffer, the compiler's auto-vectorizer can turn it into SIMD instructions that process several pixels per cycle.

Flynn's taxonomy. The two opportunities correspond to two classes of Flynn's taxonomy. Multithreading is *MIMD* (Multiple Instruction, Multiple Data): each thread runs on its own core with its own instruction stream, on its own tiles, and may be culling one tile while another thread is blending pixels of a different one. Vectorization is *SIMD* (Single Instruction, Multiple Data): a single instruction operates on several pixels at once, all following the same path. The four kernels therefore cover every combination: `seq` is SISD, `pixels` and `tiles` are MIMD only, and `simd` on several threads combines MIMD across cores with SIMD within each core.

3. Parallel Implementation

Four rendering kernels are implemented and compiled just-in-time with Numba, which generates native code through LLVM (`@njit`); the three parallel ones run on a pool of Python threads on the free-threaded build of Python 3.14 (§3.2). They are referred to as `seq`, `pixels`, `tiles` and `simd`, and all four are required to produce the same image (up to floating-point reassociation error from `fastmath`).

3.1. Strategy Overview

Mode	Parallelism	Notes
`seq`	none	baseline, bbox scan
`pixels`	threads over rows	no culling, slow
`tiles`	threads over tiles	bbox culling per tile
`simd`	tiles + vector inner loop	fastest, branchless

Table 1: The four rendering kernels compared in this project.

`pixels` parallelizes over image rows: each thread owns a disjoint set of rows and, for every pixel, loops over *all* N circles with a cheap bounding-box rejection test. It has no spatial culling structure, so it is included mainly as a "multithreading alone" reference point — it is intentionally slow.

`tiles` instead parallelizes over square tiles of the image. For each tile, a thread-local candidate list is built once by testing which circles' bounding boxes intersect the tile; only that (usually much smaller) candidate set is then scanned against every pixel of the tile. Disjoint tiles mean disjoint writes, so again no synchronization is needed.

`simd` uses the same tile/culling structure as `tiles`, but the inner double loop over a tile's pixels is restructured for auto-vectorization: for each candidate circle, all pixels of the tile are visited in a countable, branch-free loop that computes a $0/1$ mask instead of using an `if`, and pixels are accumulated into small contiguous per-tile R/G/B scratch buffers before being written back to the image once at the end:

```
for ii in range(tw): # vectorizable
   d2 = dx*dx + dy2 # FMA under fastmath
   m = 1.0 if d2 <= r2 else 0.0 # branchless mask
   rr[idx] = m*(psr + rr[idx]*inv) + (1-m)*rr[idx
       ]
```

Removing the branch and keeping the accumulator arrays contiguous and small (tile-sized) lets LLVM auto-vectorize the loop with unit-stride vector loads/stores, and lets `fastmath=True` fuse the distance computation into fused multiply-adds.

3.2. Multithreading on Free-Threaded Python

CPython's Global Interpreter Lock (GIL) allows only one thread at a time to execute Python code, so on a classic interpreter plain Python threads cannot speed up CPU-bound work: they take turns on a single core. PEP 703 introduced an optional *free-threaded* build of CPython in which the GIL is removed; it is experimental in Python 3.13 and officially supported in Python 3.14 (`python3.14t`) [1]. On

this build, threads created with the standard `threading` and `concurrent.futures` APIs run truly in parallel on different cores, as in C or Java. Numba supports the free-threaded build since version 0.65, so the compiled kernels can be called concurrently from several threads.

The project runs on Python 3.14t, managed with `uv`, and the benchmark script refuses to start if the GIL is active. Running the same code on the same interpreter with the GIL re-enabled (`python -X gil=1`) is used as a control experiment (§5.7).

The parallel versions are built on this mechanism. Each kernel receives two extra arguments, `start` and `step`, and processes only the work units (rows for `pixels`, tiles for `tiles` and `simd`) with index `start`, `start+step`, `start+2*step`, ... The Python wrapper submits one call per thread to a `concurrent.futures.ThreadPoolExecutor` with p workers (thread k gets `start`$= k$, `step`$= p$) and then waits for all of them, which acts as the final barrier:

```
@njit(cache=True, fastmath=True)
def kernel(img, ..., start, step):
   for t in range(start, n_tiles, step):
      ... # render tile t

futures = [pool.submit(kernel, *args, k, p)
        for k in range(p)]
for f in futures:
   f.result() # join
```

The pools are created once per thread count and reused, so thread creation is not part of the measured time. With $p = 1$ the kernel is called directly on the main thread. The kernel code itself is the same as a sequential loop over the work units; only the loop bounds change.

Work distribution. Because circle radii vary widely, the amount of work per tile is highly non-uniform (a tile with only small, sparse circles finishes almost instantly; a tile covered by several large overlapping circles is far more expensive). Assigning each thread one contiguous block of tiles would give some threads all the expensive regions. The interleaved assignment above corresponds to OpenMP's `schedule(static,1)`: the tiles of every region of the image are spread across all threads, so cheap and expensive tiles average out. The load is further balanced by using many more tiles than threads, which is why the tile size is studied explicitly in Section 5.

3.3. Synchronization

By construction, every kernel here operates on disjoint output regions (rows or tiles), so the renderer itself is **lock-free**: no shared mutable accumulator is ever written by more than one thread, and no locks, atomics or critical sections are required.

3.4. Data Structures and Memory Layout

The default layout is SoA, as described above. For comparison, a `to_aos()` conversion produces an interleaved $(N, 8)$ *Array-of-Structures* layout, used only by the memory-layout benchmark (Section 5.5); the renderer itself never uses it internally.

4. Experimental Setup

4.1. Environment

Benchmarks were executed on a lab server with an **AMD Ryzen 5 2400G** (x86-64, first-generation Zen, 3.6 GHz base clock): 4 identical physical cores with simultaneous multithreading (SMT, 2 hardware threads per core), i.e. 8 logical cores. Each core has a private 32 KB L1 data cache, the size the default tile (32×32 pixels, whose three per-tile R/G/B buffers take 12 KB) was tuned for. The CPU supports AVX2 and FMA, so the vectorized kernel operates on 256-bit vectors of 8 `float32`; note that Zen executes each 256-bit AVX2 instruction as two 128-bit halves. The machine runs Debian 13 as a virtual machine on Proxmox. The software stack was the free-threaded build of Python 3.14.8 (GIL disabled), NumPy 2.5.3, Numba 0.68.0 and llvmlite 0.50.0 on Linux, with the environment managed by `uv`. The maximum number of threads in the sweeps was deliberately set to **16**, i.e. *twice* the number of logical cores, so that the thread-scaling experiment can observe oversubscription behavior (thread count exceeding the logical core count) as required by the course guidelines [1], in addition to normal scaling up to the core count. Unlike a hybrid CPU, all cores run at the same speed, so the thread sweep has three clearly separated regimes: 1–4 threads (one thread per physical core), 5–8 threads (two threads sharing the execution units of a physical core through SMT) and 9–16 threads (oversubscription). The server is shared, so benchmarks were run while it was otherwise idle.

4.2. Dataset

The dataset is procedurally generated: N circles with uniformly random position, radius (scaled to image resolution), color and alpha. This is a deliberate choice rather than a limitation — for a 2D rasterizer, synthetically generated geometry *is* the realistic workload, in the same way standard rasterization benchmarks use synthetic scenes rather than an external image dataset; there is no natural pre-existing dataset of "circles with depth and alpha" to substitute.

4.3. Methodology and Parameters

- **Timing:** wall-clock time via `time.perf_counter` equivalents, with a per-configuration time budget that stops adding iterations once exceeded (bounding the cost of slow configurations), plus CPU time (`time.process_time()`, summed across threads) and an approximate 95% confidence interval ($1.96 \cdot \sigma/\sqrt{n}$).
- **Warm-up:** each configuration is preceded by 2 warm-up runs (excluded from timing) to absorb Numba's JIT compilation cost; kernels are also cached to disk (`@njit(cache=True)`) across runs.



Figure 1: Example output of the rasterizer: several thousand alpha-blended, depth-sorted circles composited with the "over" operator.

- **Iterations:** 5 timed iterations per configuration (subject to the time budget).
- **Scaling dataset:** the strong-scaling experiment (§5.1) deliberately uses a large problem (2048^2 pixels, 60,000 circles) so that the sequential baseline takes ≥ 10 s, as required by the course's benchmarking guidelines [1] for meaningful speedup measurements.
- **Validation:** before any timing, the benchmark script renders the same scene with every kernel (`pixels`, `tiles`, `simd` and the AoS variant) at the maximum thread count and compares each image with the sequential one. The check uses a tolerance of 10^{-3} on the maximum absolute pixel difference rather than bit-exact equality, since `fastmath=True` permits floating-point reassociation, and it stops the run if any kernel exceeds it. Two cases are tested: a 512^2 image and a 1000×700 image, whose sides are not multiples of the tile size (partial tiles at the borders). The largest difference measured was [$\sim 10^{-7}$], i.e. float32 rounding.

5. Results and Analysis

5.1. Strong Scaling

Figure 2 shows time, speedup and parallel efficiency versus thread count for a fixed, large problem (2048^2 pixels, 60,000 circles; sequential baseline: [T_{seq}] s). Table 2 reports the same data for the `simd` kernel.

Three effects are visible. First, vectorization alone already gives the `simd` kernel a large head start at p=1 ([X]× faster than `seq` on a single thread), confirming that SIMD and multithreading are largely independent, additive sources of speedup. Second, **[scaling over 1–4 threads, one thread per physical core]**. Third, **[gain from 5–8 threads, where SMT siblings share a core's execution units, and behavior beyond 8 threads, under oversubscription]**. The `tiles` kernel (culling, no SIMD) is **[position relative to `pixels` and `simd`]**: culling alone (`tiles`) reaches [X]× at best, while adding SIMD on top pushes the maximum speedup to [S_{max}]. On this dense workload (60,000 large circles) each tile has a long candidate list, so `tiles` tests many circles per pixel, whereas `seq` only visits the pixels inside each circle's bounding box.

Threads	Time (ms)	Speedup	Efficiency
1	[]	[]	[]
2	[]	[]	[]
4	[]	[]	[]
8	[]	[]	[]
16	[]	[]	[]

Table 2: Strong scaling of the `simd` kernel on the 60,000-circle, 2048^2 workload (speedup relative to the [T_{seq}] s sequential baseline; efficiency here is speedup/p).

5.2. Effect of Resolution and Circle Count

Figures 3–4 sweep image resolution and circle count independently (at a fixed thread count of 16). Both scale roughly linearly with problem size for all three kernels tested (`seq`, `tiles`, `simd`), consistent with the algorithm's cost being dominated by pixels-inside-bounding-boxes rather than by a resolution- or count-dependent constant overhead. `simd` is always the fastest, [X–Y]× faster than `tiles`, with the gap growing with problem size. `tiles` beats `seq` in most configurations (at 2048^2, `tiles` is [X]× faster than `seq`; `simd` is [X]× faster), but falls behind it at 8000 circles ([ms] vs. [ms]): with many circles the per-tile candidate lists grow long, the same effect seen in the strong-scaling experiment.

5.3. Effect of Tile Size

Figure 5 sweeps the tile side length from 16 to 256 pixels (1024^2 image, 4000 circles, 16 threads). `simd` is fastest at **tile = 32**, the value used as the default everywhere else, and `tiles` at tile = [N]; both degrade quickly for larger tiles: bigger tiles mean fewer, coarser parallel work units (worse load balancing and less L1/L2 residency for the per-tile scratch buffers), while very small tiles pay a proportionally larger fixed per-tile overhead (building the candidate list). At tile = 32 the three R/G/B scratch buffers of `simd` take 12 KB and fit entirely in the 32 KB L1 data cache; at tile = 64 they take 48 KB and no longer do. `simd` is consistently [X–Y]× faster than `tiles` across the whole sweep.

5.4. Effect of SIMD (Scalar vs. Vectorized)

To isolate the contribution of vectorization from thread count, Figure 6 compares `tiles` (scalar inner loop) against `simd` (vectorized inner loop) across the same thread sweep (1–16),

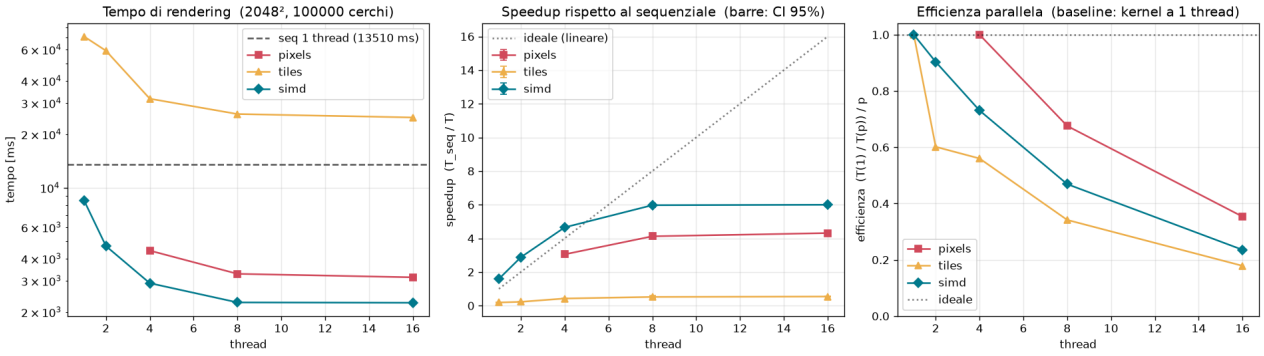


Figure 2: Strong scaling: rendering time (log scale), speedup vs. the sequential baseline, and parallel efficiency, for all three parallel kernels, from 1 to 16 (oversubscribed) threads.

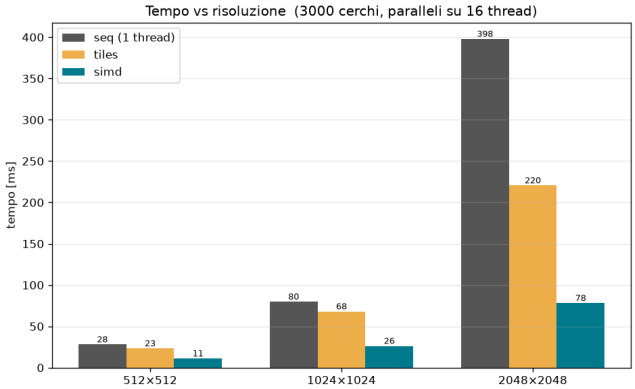


Figure 3: Rendering time vs. image resolution (3000 circles, 16 threads).

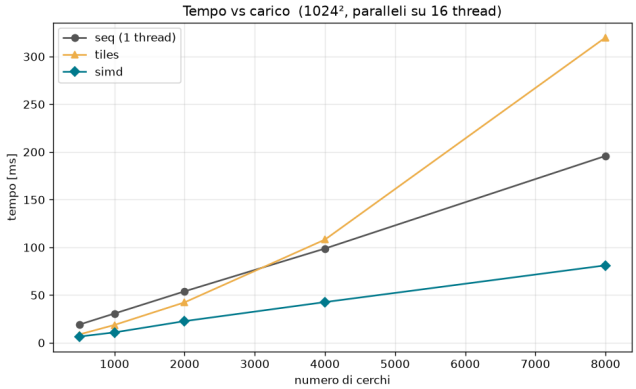


Figure 4: Rendering time vs. circle count (1024^2 image, 16 threads).

at fixed resolution and circle count. At every thread count `simd` is faster: **[X]**× at p=1 (**[ms]** vs. **[ms]**) and still **[X]**× at p=16 (**[ms]** vs. **[ms]**). A 256-bit AVX2 vector holds 8 `float32` lanes, but first-generation Zen splits each 256-bit operation into two 128-bit halves, so per cycle it processes about as many lanes as 128-bit SSE (4 `float32`). **[comment on how the measured ratio compares with this]**. In Flynn's terms, this experiment separates the two forms of parallelism: at p=1 `tiles` is SISD and `simd` is SIMD only; along the thread axis `tiles` is MIMD only, while `simd` combines MIMD and SIMD. This confirms that vectorization and multithreading are two genuinely separate levers on performance, and that neither substitutes for the other.

5.5. Memory Layout: SoA vs. AoS

Figure 7 compares the default SoA layout against an interleaved AoS layout for the otherwise-identical `tiles` kernel. The `tiles` kernel was chosen over `simd` on purpose: in `tiles`, circle attributes are read inside the per-pixel loop (for every candidate circle at every pixel), so it is the kernel most sensitive to how circles are laid out in memory. In `simd`, instead, each circle's attributes are loaded once per tile, before the vectorized loop, which then only touches the per-tile R/G/B buffers; since SIMD there works across pixels, not across circles, the circle layout would have almost no effect. **[SoA vs. AoS times at 512^2, 1024^2, 2048^2 and whether either layout wins consistently]**. This is expected for this workload: circle attributes are read once per tile-candidate-test and once per pixel hit, not streamed sequentially over the whole array in a tight loop, so the unit-stride advantage of SoA is largely hidden by the small candidate-list sizes and by cache residency of the per-tile working set — the memory-layout choice matters far less here than it would for a bulk, streaming numerical kernel.

5.6. Weak Scaling

Figure 8 keeps work-per-thread constant by growing the circle count proportionally to the thread count (500 circles per thread). Efficiency is **[values at 1, 2, 4, 8, 16 threads]**. **[interpretation of the 1–4, 5–8 (SMT) and 9–16 (oversubscription) regimes, and whether it matches strong scaling]**.

5.7. GIL vs. Free-Threading

To verify that the measured speedup really comes from the absence of the GIL, the `simd` kernel (1024^2 image, 4000 circles) is timed over the same thread sweep (1–16) in two subprocesses launched with the *same* free-threaded interpreter:

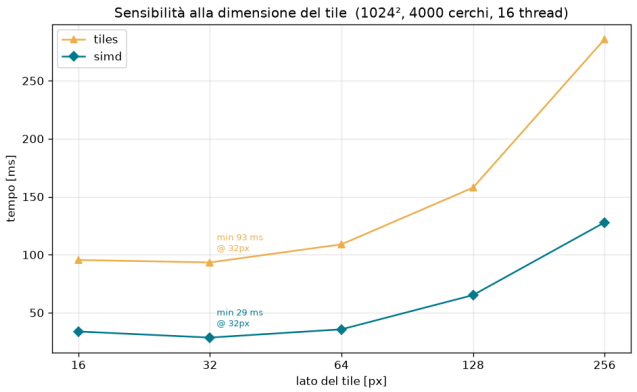


Figure 5: Rendering time vs. tile size, for the `tiles` and `simd` kernels.

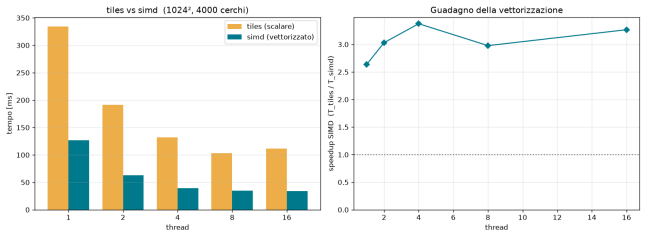


Figure 6: Scalar (`tiles`) vs. vectorized (`simd`) kernel, across thread counts 1–16 (1024^2 image, 4000 circles).

one with `-X gil=0` (free-threading, the configuration used everywhere else) and one with `-X gil=1` (GIL re-enabled at startup). Code, data, interpreter and machine are identical; only the GIL changes. The benchmark script also checks, inside each subprocess, that the GIL is actually in the requested state.

The kernels are compiled without Numba's `nogil=True`, so with the GIL enabled a thread holds the lock for the whole duration of its kernel call and the other threads wait: they run one at a time, and no speedup is expected. Figure 9 confirms this: **[speedup with the GIL stays at about X× for every thread count, while without the GIL it reaches Y× at p threads]**. This shows that the parallelism measured in this report is due to the free-threaded build of CPython, not to the compiled code releasing the GIL on its own.

5.8. Comparison with Amdahl's and Gustafson's Laws

Amdahl's law [2] bounds the speedup on p threads as $S(p) = 1/\big(f + (1-f)/p\big)$, where f is the fraction of the run that cannot be parallelized. In the `simd` wrapper the only serial work is allocating the image and sorting the circles by Z: about **[ms]** out of the **[s]** single-thread run on the strong scaling workload, so $f \approx$**[f]**. With this f, Amdahl predicts $S(8) \approx$**[]** and $S(16) \approx$**[]** relative to `simd` on one thread. The measured values (Table 2) are **[values at 8 and 16 threads]**. The Karp–Flatt metric $e = \big(1/S(p) - 1/p\big)/\big(1 - 1/p\big)$, which estimates the serial fraction from the measured speedup, gives

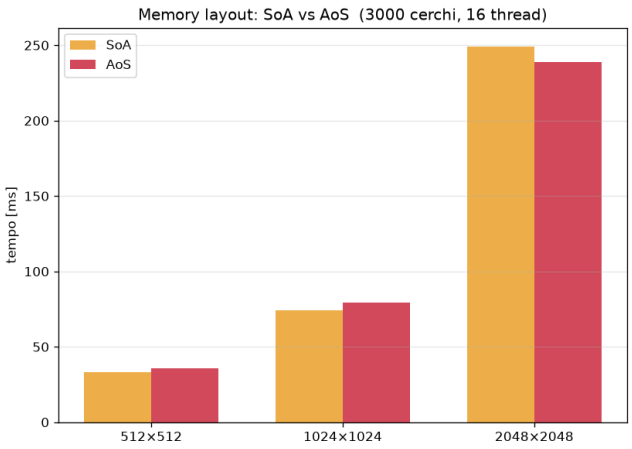


Figure 7: Rendering time, SoA vs. AoS circle storage, across resolutions.

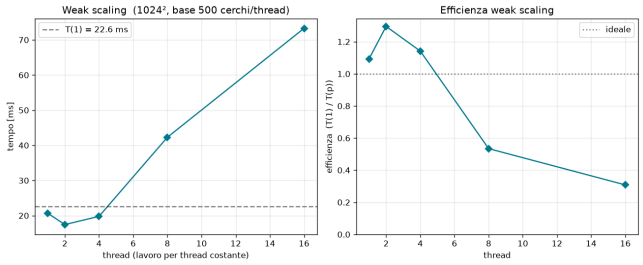


Figure 8: Weak scaling: circle count grows proportionally with thread count (500 circles/thread); efficiency relative to the 1-thread baseline.

$e =$**[]** for $p = 2, 4, 8, 16$. **[interpretation of e: SMT from 5 threads on, oversubscription beyond 8]**.

Gustafson's law [3] describes the weak scaling case, where the problem grows with p: the scaled speedup is $S(p) = p - f\,(p-1)$, so with $f \approx$**[f]** the efficiency should stay close to 1. **[comparison with the measured weak scaling efficiency (Fig. 8)]**.

5.9. Bottleneck Analysis

Taken together, the experiments point to two dominant, largely independent limits on performance:

- **Physical core count, SMT and oversubscription.** **[what strong and weak scaling show in the three regimes: 1–4 threads (one per physical core), 5–8 threads (SMT siblings competing for the same execution units, which a compute-bound vectorized kernel already keeps busy) and beyond 8 threads (oversubscription of the 8 logical cores, scheduling overhead only)]**.
- **Load imbalance from circle-size variance**, which is exactly why the interleaved tile distribution and tile-size tuning matter more than memory-layout choices for this workload: unlike a dense numerical kernel, the bottleneck here is uneven *work* distribution across parallel

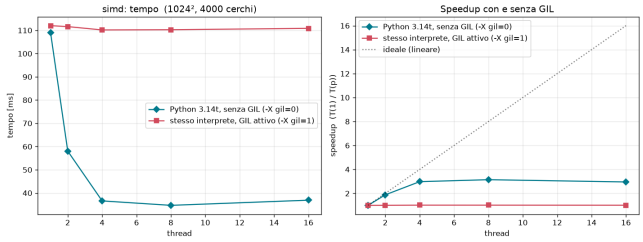


Figure 9: Speedup of the `simd` kernel vs. thread count on the same Python 3.14t interpreter, with the GIL disabled (`-X gil=0`) and re-enabled (`-X gil=1`); 1024^2 image, 4000 circles.

units, not memory bandwidth or cache locality (as confirmed by the lack of a consistent SoA/AoS difference).

6. Conclusions

This project implemented and systematically benchmarked a CPU-parallel 2D circle rasterizer, combining coarse-grained multithreading (Python threads on free-threaded Python 3.14, over image tiles) with fine-grained SIMD vectorization (a branchless, auto-vectorizable inner loop). The tiled, vectorized kernel reaches a maximum speedup of **[S_{max}]** over the sequential baseline on a 4-core / 8-thread AMD Ryzen 5 2400G, with vectorization alone contributing roughly **[X$\times$]** independently of thread count. Culling (tile-based candidate lists) and vectorization proved to be complementary, additive optimizations rather than substitutes for one another. A dedicated tile-size sweep identified small tiles (32 px for `simd`) as the sweet spot for this workload, and, since tiles are disjoint, the renderer needs no synchronization at all. Memory layout (SoA vs. AoS) had a comparatively minor effect, consistent with a workload whose bottleneck is irregular per-tile work rather than memory bandwidth. **[one-sentence summary of strong and weak scaling across the physical-core, SMT and oversubscription regimes]**.

Future work could explore spatial acceleration structures (e.g. a tile-local BVH or grid over circles for very high circle counts), GPU offload of the tile-based kernel, and adaptive tile sizing driven by local circle density rather than a single fixed tile size.

References

[1] Parallel Programming course, Università degli Studi di Firenze, (course slides: assignment descriptions, implementation guidelines and technical report structure).

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *Proc. AFIPS Spring Joint Computer Conference*, 1967.

[3] R. Robey, Y. Zamora, *Parallel and High Performance Computing*, Manning, 2021.

Authorship: I confirm that I am the sole author of this writeup and the associated code.